Linux File Permissions

File permissions are the backbone of Linux security — and since privilege escalation (a core pentesting skill) almost always involves *abusing* misconfigured permissions, understanding this deeply is non-negotiable for your path.

---

1. The Basics — Reading `ls -l` Output

```
$ ls -l
-rwxr-xr-- 1 omkar staff 4096 Jul 11 10:00 script.sh
```

Breaking down `-rwxr-xr--`:

```
-    rwx    r-x    r--
|     |      |      |
type  owner  group  others
```

Position 1 — File type:

Symbol	Meaning
`-`	Regular file
`d`	Directory
`l`	Symbolic link
`c`	Character device
`b`	Block device
`s`	Socket
`p`	Named pipe

Positions 2-10 — Three permission triads:

- Characters 2-4 → **Owner** permissions
- Characters 5-7 → **Group** permissions
- Characters 8-10 → **Others** (everyone else) permissions

---

2. The Three Permission Types

Symbol	Permission	Effect on a File	Effect on a Directory
`r`	Read	View file contents	List directory contents (`ls`)
`w`	Write	Modify/delete file contents	Create/delete/rename files inside it
`x`	Execute	Run the file as a program/script	`cd` into the directory

Important nuance: on a directory, `r` without `x` lets you list filenames but not access details (like via `stat`), and `x` without `r` lets you `cd` into it and access known filenames but not list contents — this distinction shows up often in privilege escalation scenarios.

---

3. Numeric (Octal) Notation

Each permission has a numeric value:

```
r = 4
w = 2
x = 1
```

You add them per triad:

Combo	Value	Meaning
rwx	7	read+write+execute
rw-	6	read+write
r-x	5	read+execute
r--	4	read only
-wx	3	write+execute
-w-	2	write only
--x	1	execute only
---	0	no permissions

So `-rwxr-xr--` = **754**:

- Owner: rwx = 7

- Group: r-x = 5
- Others: r-- = 4

Common permission sets you'll see constantly:

- `755` — owner full control, everyone else can read/execute (typical for scripts/binaries)
- `644` — owner read/write, everyone else read-only (typical for regular files)
- `700` — owner full control, no one else has any access (private scripts/keys)
- `777` — everyone has full control (a huge red flag in any security audit)

---

4. Changing Permissions — `chmod`

Numeric method:

```
chmod 755 script.sh
chmod 644 notes.txt
chmod 700 private_key.pem
```

Symbolic method — often more precise for small tweaks:

```
chmod u+x script.sh      # add execute for owner (user)
chmod g-w file.txt       # remove write for group
chmod o=r file.txt       # set others to read-only exactly
chmod a+r file.txt       # add read for all (owner+group+ot
hers)
chmod ug+rw file.txt     # add read+write for owner and gro
up
```

Symbol	Meaning
`u`	user (owner)
`g`	group
`o`	others
`a`	all (u+g+o)
`+`	add permission
`-`	remove permission

Symbol	Meaning
`=`	set exactly (overwrites existing)

Recursive (apply to a directory and everything inside it):

```
chmod -R 755 /var/www/html
```

---

5. Changing Ownership — `chown` and `chgrp`

```
chown omkar file.txt              # change owner
chown omkar:staff file.txt        # change owner AND group
chgrp staff file.txt              # change group only
chown -R omkar:staff /home/omkar/project   # recursive
```

Only **root** (or the current owner, in limited cases) can change ownership — this is a key distinction from `chmod`, which the owner can always do on their own files.

---

6. Special Permissions (the ones that matter most for pentesting)

Beyond the standard `rwx`, there are three special bits that show up constantly in privilege escalation.

SUID (Set User ID) — `s` in owner's execute position

When set on an executable, the program runs with the **file owner's** privileges, not the user who runs it.

```
ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 68208 /usr/bin/passwd
```

That `s` instead of `x` in the owner slot means: any user who runs `passwd` executes it **as root**, temporarily — which is exactly why regular users can change their own password even though `/etc/shadow` is root-only.

Set it:

```
chmod u+s file      # symbolic
chmod 4755 file     # numeric — the leading 4 sets SUID
```

Why this matters for you: finding SUID binaries is one of the first things you do in Linux privilege escalation:

```
find / -perm -4000 -type f 2>/dev/null
```

If you find a non-standard SUID binary (like a custom script or a binary like `find`, `vim`, `python` with SUID set), it's often a direct path to root — this is a classic technique tested on GTFOBins (gtfobins.github.io), which catalogs exactly how to abuse SUID/sudo misconfigurations on common binaries.

SGID (Set Group ID) — `s` in group's execute position

Similar to SUID, but the program/directory runs with the **group's** privileges.

On a **directory**, SGID has a different, very useful effect: any new file created inside inherits the directory's group (instead of the creating user's primary group) — commonly used for shared team directories.

```
chmod g+s /shared/project    # symbolic
chmod 2755 /shared/project   # numeric — leading 2 sets SGI
D
```

Finding SGID binaries:

```
find / -perm -2000 -type f 2>/dev/null
```

Sticky Bit — `t` in others' execute position

Mostly used on **directories**. It means: even if a user has write access to the directory, they can only delete/rename **their own files**, not other users' files.

```
ls -ld /tmp
drwxrwxrwt 20 root root 4096 Jul 11 10:00 /tmp
```

This is why `/tmp` is world-writable (`777`) but users can't delete each other's temp files — the sticky bit `t` at the end protects that.

```
chmod +t /shared/uploads     # symbolic
chmod 1777 /shared/uploads   # numeric — leading 1 sets sti
cky bit
```

7. Combining Special Permission Digits

The numeric format for special bits is a **4th leading digit**:

```
chmod 4755 file   → SUID + rwxr-xr-x
chmod 2755 file   → SGID + rwxr-xr-x
chmod 1777 dir    → Sticky + rwxrwxrwx
chmod 6755 file   → SUID + SGID + rwxr-xr-x  (4+2=6)
```

8. Default Permissions — `umask`

New files/directories don't start at `777` / `666` — the **umask** subtracts permissions by default.

```
umask
0022
```

- Default max for files: `666` (no execute by default, for safety)
- Default max for directories: `777`

With umask `022`, new files get `666 - 022 = 644`, new directories get `777 - 022 = 755`.

Change it (usually in `.bashrc` for persistence):

```
umask 027    # more restrictive — group gets read-only, oth
ers get nothing
```

9. Access Control Lists (ACLs) — Going Beyond owner/group/other

Standard permissions only let you define access for exactly 3 entities. ACLs let you grant fine-grained permissions to *specific* additional users/groups.

```
getfacl file.txt              # view ACLs
setfacl -m u:omkar:rw file.txt   # give user 'omkar' read+w
rite
setfacl -m g:devteam:r file.txt  # give group 'devteam' rea
d
setfacl -x u:omkar file.txt      # remove that specific ACL
entry
```

When a file has extra ACLs, `ls -l` shows a `+` at the end of the permission string:

```
-rw-rw-r--+ 1 omkar staff 1024 file.txt
```

---

10. Practical Privilege Escalation Angle (Study Context)

Since this connects directly to your VAPT work, here's how file permissions become an attack surface:

a) Writable `/etc/passwd` or `/etc/shadow`

```
ls -l /etc/passwd
find / -writable -type f 2>/dev/null | grep -E "passwd|shad
ow"
```

If writable by a low-priv user, you can add a new root-equivalent user entry directly.

b) SUID binaries not meant to have it

```
find / -perm -4000 -type f 2>/dev/null
```

Cross-reference results against GTFOBins to see if any known binary can be abused to spawn a root shell.

c) Writable cron job scripts
If a cron job owned by root executes a script that's writable by your low-priv user, you can inject a reverse shell into that script and wait for cron to run it as root.

```
find / -writable -type f 2>/dev/null | xargs ls -la 2>/dev/
null | grep root
```

d) Sudo misconfigurations (related, checked with `sudo -l`)
Not technically file permissions, but always checked alongside — shows what commands your user can run as root without a password, which combined with GTFOBins is often an instant win.

Quick Reference Cheat Sheet

```
chmod 755 file          # rwxr-xr-x
chmod 644 file          # rw-r--r--
chmod +x file           # add execute for everyone
chmod u+s file          # SUID
chmod g+s dir           # SGID
chmod +t dir            # sticky bit
chown user:group file   # change owner+group
find / -perm -4000 2>/dev/null   # find all SUID binaries
(classic first move on any Linux box)
```
